

WYKŁAD 11

Tkinter — interfejsy graficzne

Kurs języka Python — semestr 2025/2026

Wprowadzenie do programowania II | Informatyka Stosowana

Akademia Nauk Stosowanych w Nowym Sączu

Tematy

1	Tkinter - wprowadzenie	moduł wbudowany, Tk(), mainloop(), pierwszy program
2	Widżety podstawowe	Label, Button, Entry, Text, Check/Radio
3	Menedżery układu	pack(), grid(), place() - pozycjonowanie widżetów
4	Zmienne Tk i zdarzenia	StringVar, IntVar, command, bind(), protocol
5	Menu i okna dialogowe	Menubar, messagebox, filedialog, Toplevel
6	Canvas	rysowanie figur, tekst, interakcja, animacja
7	ttk - nowoczesne widżety	ttk vs tk, Treeview, Combobox, Style
8	Mini-projekt	kompletna aplikacja GUI - klasa, layout, logika
9	Dobre praktyki	architektura, separacja logiki, testowalność

Tkinter - wprowadzenie

Wbudowany moduł do tworzenia aplikacji okienkowych

Czym jest Tkinter?

Standardowa biblioteka GUI w Pythonie - nie wymaga instalacji

Właściwości

- Wbudowany w Pythona - import tkinter
- Nakładka na bibliotekę Tcl/Tk
- Cross-platform: Windows, macOS, Linux
- Prosty w nauce - idealny na start z GUI
- Wbudowane widżety, layout, zdarzenia
- Alternatywy: PyQt, PySide, wxPython, Kivy

Pierwszy program

```
import tkinter as tk

root = tk.Tk()
root.title("Moja aplikacja")
root.geometry("400x300")

label = tk.Label(root,
                  text="Witaj w Tkinterze!")
label.pack(pady=20)

root.mainloop()
# ^^^ blokuje – pętla zdarzeń
```

✓ Konwencja: import tkinter as tk - krótki alias, stosowany powszechnie.

Anatomia aplikacji Tk

Hierarchia: root → ramki → widgety → mainloop

Schemat działania

1. Utwórz root

`root = tk.Tk()`

2. Konfiguruj okno

`title()`, `geometry()`, `resizable()`

3. Dodaj widgety

`Label`, `Button`, `Entry`, `Frame...`

4. Rozmieść widgety

`pack()`, `grid()` lub `place()`

5. Uruchom pętlę

`root.mainloop()`

Konfiguracja okna

```
root = tk.Tk()

# Tytuł i rozmiar
root.title("Aplikacja")
root.geometry("600x400")      # szer x wys
root.geometry("600x400+100+50")
#                ^^^ pozycja x,y

# Ograniczenia rozmiaru
root.minsize(300, 200)
root.maxsize(1200, 800)
root.resizable(True, False)  # szer, wys

# Ikona i tło
root.iconbitmap("ikona.ico")  # Windows
root.configure(bg="#f0f0f0")

# Zamknięcie okna
root.protocol("WM_DELETE_WINDOW",
              on_close)
```

Widgety podstawowe

Label, Button, Entry, Text, Checkbutton, Radiobutton

Label, Button, Entry

Trzy najczęściej używane widgety

Label - etykieta tekstowa

```
lbl = tk.Label(root,  
    text="Imię:",  
    font=("Arial", 14),  
    fg="navy",           # kolor tekstu  
    bg="#f0f0f0",       # tło  
    anchor="w")         # wyrównanie  
lbl.pack(padx=10, pady=5)
```

Button — przycisk

```
def on_click():  
    print("Kliknięto!")  
  
btn = tk.Button(root,  
    text="Kliknij mnie",  
    command=on_click,    # callback!  
    width=20, height=2,  
    bg="steelblue", fg="white")  
btn.pack(pady=10)
```

Entry - pole tekstowe (jednoliniowe)

```
entry = tk.Entry(root, width=30, font=("Arial", 12))  
entry.pack(pady=5)  
entry.insert(0, "Tekst domyślny")           # wstaw tekst na początku  
  
# Odczyt i czyszczenie  
tekst = entry.get()                         # pobierz wartość  
entry.delete(0, tk.END)                     # wyczyść całość  
  
# Hasło (ukryte znaki)  
passwd = tk.Entry(root, show="*")           # wyświetla gwiazdki zamiast tekstu
```

Text - wieloliniowy edytor tekstu

Edytor z obsługą tagów, przewijaniem i formatowaniem

Tworzenie i obsługa

```
txt = tk.Text(root,
    width=40,      # znaki
    height=10,     # wiersze
    font=("Consolas", 11),
    wrap=tk.WORD)  # zawijanie słów
txt.pack(padx=10, pady=10)

# Wstaw tekst
txt.insert(tk.END, "Witaj!\n")
txt.insert("1.0", "Na początku\n")

# Odczyt
tekst = txt.get("1.0", tk.END)
# "1.0" = wiersz 1, znak 0

# Czyszczenie
txt.delete("1.0", tk.END)

# Tylko do odczytu
txt.config(state=tk.DISABLED)
```

Scrollbar + Text

```
frame = tk.Frame(root)
frame.pack(fill=tk.BOTH, expand=True)

scroll = tk.Scrollbar(frame)
scroll.pack(side=tk.RIGHT,
    fill=tk.Y)

txt = tk.Text(frame,
    yscrollcommand=scroll.set)
txt.pack(side=tk.LEFT,
    fill=tk.BOTH, expand=True)

scroll.config(command=txt.yview)

# Tagi – formatowanie fragmentów
txt.tag_configure("bold",
    font=("Arial", 12, "bold"))
txt.tag_configure("red",
    foreground="red")

txt.insert(tk.END, "Ważne!", "bold")
txt.insert(tk.END, " Błąd!", "red")
```


Checkbox i Radiobutton

Checkbox - wielokrotny wybór

```
# Zmienna śledząca
bold_var = tk.BooleanVar()
italic_var = tk.BooleanVar()

cb1 = tk.Checkbutton(root,
    text="Pogrubienie",
    variable=bold_var,
    command=update_font)
cb1.pack(anchor="w")

cb2 = tk.Checkbutton(root,
    text="Kursywa",
    variable=italic_var,
    command=update_font)
cb2.pack(anchor="w")

# Odczyt stanu
if bold_var.get():
    print("Pogrubienie ON")
```

Radiobutton - jeden z wielu

```
# Wspólna zmienna – klucz!
color_var = tk.StringVar(value="red")

colors = [("Czerwony", "red"),
    ("Zielony", "green"),
    ("Niebieski", "blue")]

for text, val in colors:
    rb = tk.Radiobutton(root,
        text=text,
        variable=color_var,
        value=val,
        command=on_color_change)
    rb.pack(anchor="w")

# Odczyt wybranej wartości
wybrany = color_var.get() # "red"

# Zmiana programowa
color_var.set("blue")
```

- ✓ Radiobuttons działają grupami - wszystkie w grupie muszą dzielić tę samą zmienną (variable).

Listbox, Combobox, Spinbox, Scale

Listbox

```
lb = tk.Listbox(root, height=5,
               selectmode=tk.MULTIPLE)
for item in ["Python", "Java", "C++"]:
    lb.insert(tk.END, item)
lb.pack()

# Odczyt zaznaczenia
sel = lb.curselection() # krotka idx
for i in sel:
    print(lb.get(i))
```

Combobox (ttk) i Spinbox

```
from tkinter import ttk

cb = ttk.Combobox(root,
                 values=["mały", "średni", "duży"],
                 state="readonly") # bez edycji
cb.set("średni") # domyślny
cb.pack()
cb.bind("<<ComboboxSelected>>",
       on_select)

# Spinbox
sp = tk.Spinbox(root, from_=0, to=100,
               increment=5, width=10)
```

Scale — suwak

```
sc = tk.Scale(root, from_=0, to=255, orient=tk.HORIZONTAL,
              label="Jasność:", length=300, command=on_scale_change)
sc.set(128) # wartość początkowa
sc.pack(pady=10)

val = sc.get() # odczyt aktualnej wartości (int lub float)
# command= otrzymuje wartość jako string – trzeba skonwertować: int(val)
```

Menedżery układu

`pack()`, `grid()`, `place()` - pozycjonowanie widgetów w oknie

pack() - układ liniowy

Najprostszy menedżer - pakuje widżety jeden za drugim

```
# Domyślnie: układa od góry do dołu
btn1 = tk.Button(root, text="Góra")
btn1.pack()                                # domyślne: side=tk.TOP

btn2 = tk.Button(root, text="Dół")
btn2.pack(side=tk.BOTTOM)

btn3 = tk.Button(root, text="Lewo")
btn3.pack(side=tk.LEFT)

btn4 = tk.Button(root, text="Prawo")
btn4.pack(side=tk.RIGHT)

# Kluczowe parametry
widget.pack(
    side=tk.TOP,           # TOP, BOTTOM, LEFT, RIGHT
    fill=tk.X,             # NONE, X, Y, BOTH – rozciąganie
    expand=True,           # czy zajmuje dodatkową przestrzeń
    padx=10, pady=5,       # zewnętrzny margines (piksele)
    ipadx=5, ipady=3,       # wewnętrzny padding
    anchor="w"             # n, s, e, w, center, nw, ne...
)

# Typowy wzorzec: rozciągnięty Text
txt = tk.Text(root)
txt.pack(fill=tk.BOTH, expand=True)        # wypełnia całe okno
```

grid() - układ tabelaryczny

Najpopularniejszy menedżer - wiersze i kolumny jak w tabeli

Podstawy grid

```
# Formularz logowania
tk.Label(root, text="Login:").grid(
    row=0, column=0, sticky="e",
    padx=5, pady=5)
tk.Entry(root).grid(
    row=0, column=1, padx=5, pady=5)

tk.Label(root, text="Hasło:").grid(
    row=1, column=0, sticky="e",
    padx=5, pady=5)
tk.Entry(root, show="*").grid(
    row=1, column=1, padx=5, pady=5)

tk.Button(root, text="Zaloguj").grid(
    row=2, column=0, columnspan=2,
    pady=10)

# sticky = "nsew" – jak anchor
# n,s,e,w = góra,dół,pravo,lewo
# "ew" = rozciągnij poziomo
```

Rozciąganie kolumn i wierszy

```
# Rozciąganie przy resize
root.columnconfigure(0, weight=1)
root.columnconfigure(1, weight=2)
root.rowconfigure(0, weight=1)
# weight=0 → stały rozmiar
# weight=1+ → proporcjonalny

# Scalanie komórek
btn.grid(row=2, column=0,
        columnspan=2) # 2 kolumny
lbl.grid(row=0, column=2,
        rowspan=3) # 3 wiersze

# Minimalny rozmiar kolumny
root.columnconfigure(0, minsize=100)

# Jednolity padding
for child in root.winfo_children():
    child.grid_configure(
        padx=5, pady=5)
```

place() i porównanie menedżerów

place() — pozycja absolutna

```
# Pozycja w pikselach
btn.place(x=50, y=100)

# Pozycja względna (0.0-1.0)
btn.place(relx=0.5, rely=0.5,
          anchor="center") # wycentrowany

# Rozmiar względny
frame.place(relx=0, rely=0,
            relwidth=1.0, relheight=0.5)
# górna połowa okna
```

Porównanie menedżerów

pack()	Liniowy	Proste układy, toolbary
grid()	Tabelaryczny	Formularze, dashboardy
place()	Absolutny	Precyzyjne, overlay

Frame — kontener na widgety

```
# Różne menedżery w różnych Frame'ach — to OK!
top_frame = tk.Frame(root)
top_frame.pack(side=tk.TOP, fill=tk.X) # pack dla ramki
tk.Button(top_frame, text="A").grid(row=0, column=0) # grid W RAMCE
tk.Button(top_frame, text="B").grid(row=0, column=1) # grid W RAMCE

bottom_frame = tk.Frame(root)
bottom_frame.pack(side=tk.BOTTOM, fill=tk.BOTH, expand=True)
```

Mieszanie menedżerów układu

pack() i grid() w tym samym kontenerze - deadlock!

❌ Problem — zamrożenie

```
# ŹŁE: pack + grid w TYM SAMYM rodzicu
btn1 = tk.Button(root, text="A")
btn1.pack()           # pack w root

btn2 = tk.Button(root, text="B")
btn2.grid(row=0)      # grid w root!

# REZULTAT:
# - Aplikacja się ZAMRAŻA
# - Oba menedżery walczą o przestrzeń
# - Nieskończona pętla negocjacji
# - Brak błędu! — po prostu wisi
```

✅ Rozwiązanie — Frame!

```
# DOBRZE: różne menedżery w różnych
# kontenerach (Frame)

frame1 = tk.Frame(root)
frame1.pack()           # pack w root

btn1 = tk.Button(frame1, text="A")
btn1.grid(row=0, column=0) # grid w frame1

frame2 = tk.Frame(root)
frame2.pack()           # pack w root

btn2 = tk.Button(frame2, text="B")
btn2.pack()             # pack w frame2
```

Zasada

W obrębie jednego kontenera (root, Frame) używaj JEDNEGO menedżera. Różne ramki mogą mieć różne menedżery.

Zmienne Tk i zdarzenia

StringVar, IntVar, command, bind(), protocol

Zmienne Tk - śledzące zmiany

StringVar, IntVar, DoubleVar, BooleanVar - powiązanie widgetu z danymi

Typy zmiennych Tk

```
# Tworzenie
name = tk.StringVar()
age = tk.IntVar(value=0)
price = tk.DoubleVar(value=9.99)
flag = tk.BooleanVar(value=False)

# Odczyt / zapis
name.set("Anna")
print(name.get())    # "Anna"

# Powiązanie z widgetem
entry = tk.Entry(root,
    textvariable=name)
entry.pack()
# Zmiany w Entry automatycznie
# aktualizują name.get() i odwrotnie!

label = tk.Label(root,
    textvariable=name)
# Label wyświetla aktualną wartość
```

trace - reakcja na zmianę

```
# Callback przy każdej zmianie
def on_name_change(*args):
    val = name.get()
    print(f"Nowa wartość: {val}")

name.trace_add("write", on_name_change)
# "write" - przy zapisie
# "read" - przy odczycie
# "unset" - przy usunięciu

# Praktyczny przykład: live search
search = tk.StringVar()
search.trace_add("write",
    lambda *a: filter_list(
        search.get()))

entry = tk.Entry(root,
    textvariable=search)
entry.pack()
# Każdy wpisany znak uruchamia
# filtrowanie listy!
```

command i bind()

Dwa sposoby reagowania na akcje użytkownika

command= (prosty callback)

```
# Funkcja bez argumentów
def save():
    print("Zapisano!")

btn = tk.Button(root, text="Zapisz",
                command=save)

# Lambda z argumentami
for i in range(5):
    tk.Button(root,
              text=f"Przycisk {i}",
              command=lambda n=i: click(n)
            ).pack()

# UWAGA: lambda n=i – konieczne!
# Bez n=i każdy przycisk wywoła
# click(4) (ostatnia wartość i)
```

bind() (zdarzenia z event)

```
# bind() – dowolne zdarzenie
def on_enter(event):
    print(f"Kliknięto: {event.x},{event.y}")

btn.bind("<Return>", on_enter) # Enter
btn.bind("<Button-1>", on_click) # LMB

# Zdarzenia klawiatury
root.bind("<Key>", on_key)
root.bind("<Control-s>", save)
root.bind("<Escape>", quit)

# Zdarzenia myszy
widget.bind("<Motion>", on_move)
widget.bind("<Enter>", on_hover)
widget.bind("<Leave>", on_leave)
widget.bind("<Double-Button-1>",
           on_dbldclick)
```

⚠ **command=** nie przekazuje event. **bind()** zawsze przekazuje event — callback musi go przyjmować.

protocol() i after() — okno i czas

protocol() — zamykanie okna

```
from tkinter import messagebox

def on_close():
    if messagebox.askokcancel(
        "Wyjście",
        "Na pewno zamknąć?"):
        root.destroy()

root.protocol("WM_DELETE_WINDOW",
             on_close)

# Bez tego – kliknięcie X zamyka
# okno bez ostrzeżenia
# root.destroy() – zamyka całą apk
# root.quit()    – kończy mainloop
```

after() — timer / animacja

```
# Jednorazowe opóźnienie (ms)
root.after(2000, show_message)
# wywoła show_message po 2 sek.

# Cykliczne wywołanie (timer)
def update_clock():
    now = time.strftime("%H:%M:%S")
    clock_label.config(text=now)
    root.after(1000, update_clock)
    # ^^ wywołuje sam siebie!

update_clock() # start

# Anulowanie
timer_id = root.after(5000, func)
root.after_cancel(timer_id)
```

⚠ Nigdy nie używaj `time.sleep()` w Tkinterze! Blokuje mainloop i zamraża okno.
Zamiast tego używaj `root.after(ms, callback)` — nie blokuje pętli zdarzeń.

Menu i okna dialogowe

Menubar, messagebox, filedialog, Toplevel

Menu i pasek menu

Menubar, Menu, kaskadowe podmenu, separatory, skróty

```
# Pasek menu
menubar = tk.Menu(root)
root.config(menu=menubar)

# Menu "Plik"
file_menu = tk.Menu(menubar, tearoff=0)          # tearoff=0 → bez linii odcinającej
menubar.add_cascade(label="Plik", menu=file_menu)

file_menu.add_command(label="Nowy",      command=new_file,   accelerator="Ctrl+N")
file_menu.add_command(label="Otwórz",    command=open_file, accelerator="Ctrl+O")
file_menu.add_command(label="Zapisz",    command=save_file, accelerator="Ctrl+S")
file_menu.add_separator()                # linia oddzielająca
file_menu.add_command(label="Wyjście",   command=root.quit)

# Skróty klawiszowe (trzeba zbindować osobno!)
root.bind("<Control-n>", lambda e: new_file())
root.bind("<Control-o>", lambda e: open_file())
root.bind("<Control-s>", lambda e: save_file())

# Menu "Edycja"
edit_menu = tk.Menu(menubar, tearoff=0)
menubar.add_cascade(label="Edycja", menu=edit_menu)
edit_menu.add_command(label="Cofnij",    command=undo)
edit_menu.add_command(label="Kopiuuj",   command=copy)

# Podmenu (kaskadowe)
export_menu = tk.Menu(file_menu, tearoff=0)
export_menu.add_command(label="PDF")
export_menu.add_command(label="CSV")
file_menu.add_cascade(label="Eksportuj", menu=export_menu)
```

Okna dialogowe

messagebox, filedialog, singledialog — gotowe okna systemowe

messagebox

```
from tkinter import messagebox

messagebox.showinfo("Info",
    "Operacja zakończona!")

messagebox.showwarning("Uwaga",
    "Plik może być duży.")

messagebox.showerror("Błąd",
    "Nie znaleziono pliku!")

# Z pytaniem (zwraca True/False)
ok = messagebox.askokcancel(
    "Potwierdzenie", "Usunąć?")

yes = messagebox.askyesno(
    "Pytanie", "Zapisać zmiany?")
```

filedialog i singledialog

```
from tkinter import filedialog
from tkinter import singledialog

# Wybór pliku
path = filedialog.askopenfilename(
    filetypes=[("CSV", "*.csv"),
               ("Wszystkie", "*.*)"]])

# Zapis pliku
path = filedialog.asksaveasfilename(
    defaultextension=".txt")

# Wybór folderu
folder = filedialog.askdirectory()

# Prosty input
name = singledialog.askstring(
    "Imię", "Podaj swoje imię:")
age = singledialog.askinteger(
    "Wiek", "Podaj wiek:", minvalue=0)
```

✓ Wszystkie dialogi są modalne — blokują interakcję z głównym oknem aż do zamknięcia.

Toplevel — dodatkowe okna

```
def open_settings():
    win = tk.Toplevel(root)           # nowe okno, potomek root
    win.title("Ustawienia")
    win.geometry("300x200")
    win.transient(root)               # zawsze nad root
    win.grab_set()                    # modalne – blokuje root

    tk.Label(win, text="Rozmiar czcionki:").pack(pady=5)
    size_var = tk.IntVar(value=12)
    tk.Spinbox(win, from_=8, to=48, textvariable=size_var, width=5).pack()

    def apply():
        font_size = size_var.get()
        # ...zastosuj zmianę...
        win.destroy()                 # zamknij okno

    btn_frame = tk.Frame(win)
    btn_frame.pack(pady=15)
    tk.Button(btn_frame, text="Zastosuj", command=apply).pack(side=tk.LEFT, padx=5)
    tk.Button(btn_frame, text="Anuluj", command=win.destroy).pack(side=tk.LEFT, padx=5)

    win.wait_window()                 # czekaj aż okno zostanie zamknięte

# Główne okno
btn = tk.Button(root, text="⚙ Ustawienia", command=open_settings)
btn.pack(pady=20)
```

Canvas

Rysowanie figur, tekst, interakcja, animacja

Canvas - rysowanie figur i tekstu

Widget do grafiki 2D - figury, linie, tekst, obrazy

Tworzenie Canvas i figury

```
c = tk.Canvas(root, width=500,
              height=400, bg="white")
c.pack()

# Prostokąt (x1,y1, x2,y2)
rect = c.create_rectangle(
    50, 50, 200, 150,
    fill="steelblue", outline="navy",
    width=2)

# Oval / koło
oval = c.create_oval(
    250, 50, 400, 150,
    fill="coral")

# Linia
line = c.create_line(
    50, 200, 400, 200,
    fill="gray", width=3, dash=(5,3))

# Wielokąt
poly = c.create_polygon(
    100,300, 200,250, 300,300,
    fill="lightgreen", outline="green")
```

Tekst, obraz, modyfikacja

```
# Tekst
txt = c.create_text(250, 30,
                    text="Tytuł", font=("Arial", 16),
                    fill="navy", anchor="n")

# Obraz (musi żyć w zmiennej!)
img = tk.PhotoImage(file="logo.png")
c.create_image(250, 200, image=img)
# BEZ ZMIENNEJ → GC usunie obraz!

# Modyfikacja istniejących obiektów
c.itemconfig(rect, fill="red")
c.coords(rect, 60, 60, 210, 160)
c.move(oval, 10, 20)    # przesun
c.delete(line)          # usuń
c.delete("all")         # usuń wszystko

# Tagi – grupowanie obiektów
c.create_oval(10,10,30,30, tags="dot")
c.create_oval(40,10,60,30, tags="dot")
c.itemconfig("dot", fill="yellow")
```

Canvas - interakcja i animacja

Zdarzenia na Canvas

```
# Klik na canvas
def on_click(event):
    c.create_oval(
        event.x-5, event.y-5,
        event.x+5, event.y+5,
        fill="red")
c.bind("<Button-1>", on_click)

# Przeciąganie obiektu (drag)
def start_drag(event):
    c._drag = c.find_closest(
        event.x, event.y)
def drag(event):
    c.coords(c._drag,
        event.x-20, event.y-20,
        event.x+20, event.y+20)
c.bind("<ButtonPress-1>", start_drag)
c.bind("<B1-Motion>", drag)
```

Animacja z after()

```
ball = c.create_oval(10,10,40,40,
    fill="blue")
dx, dy = 3, 2

def animate():
    global dx, dy
    c.move(ball, dx, dy)
    x1,y1,x2,y2 = c.coords(ball)

    if x2 > 500 or x1 < 0:
        dx = -dx # odbicie X
    if y2 > 400 or y1 < 0:
        dy = -dy # odbicie Y

    root.after(16, animate) # ~60 FPS

animate() # start animacji
```



Canvas to świetne narzędzie do gier 2D, wizualizacji, edytorów graficznych — warto go poznać.

ttk - nowoczesne widgety

Treeview, Combobox, Progressbar, Notebook, Style

ttk vs tk - nowoczesne widżety

ttk = Themed Tk — natywny wygląd systemu operacyjnego

Co daje ttk?

- Natywny wygląd (Windows, macOS, Linux)
- Treeview — drzewo / tabela danych
- Notebook — zakładki (tabs)
- Progressbar — pasek postępu
- Combobox — lista rozwijana
- Separator, Sizegrip, LabelFrame

Notebook (zakładki)

```
nb = ttk.Notebook(root)
nb.pack(fill=tk.BOTH, expand=True)

tab1 = ttk.Frame(nb)
tab2 = ttk.Frame(nb)
nb.add(tab1, text="Dane")
nb.add(tab2, text="Wykresy")

# Widżety w zakładce
tk.Label(tab1, text="Tab 1").pack()
tk.Label(tab2, text="Tab 2").pack()
```

Treeview — tabela danych

```
tree = ttk.Treeview(root, columns=("imie", "wiek", "miasto"), show="headings", height=8)
tree.heading("imie", text="Imię")
tree.heading("wiek", text="Wiek")
tree.heading("miasto", text="Miasto")
tree.column("wiek", width=60, anchor="center")
tree.insert("", tk.END, values=("Anna", 23, "Kraków"))
tree.pack(fill=tk.BOTH, expand=True)
```

ttk.Style i Progressbar

Konfiguracja wyglądu

```
style = ttk.Style()

# Motywy systemowe
print(style.theme_names())
# ('clam', 'alt', 'default', 'classic')
style.theme_use("clam")

# Własny styl przycisku
style.configure("My.TButton",
    font=("Arial", 12, "bold"),
    foreground="white",
    background="steelblue",
    padding=10)

btn = ttk.Button(root, text="Kliknij",
    style="My.TButton")
```

Progressbar

```
# Determinate – znany postęp
pb = ttk.Progressbar(root,
    orient=tk.HORIZONTAL,
    length=300, mode='determinate',
    maximum=100)
pb.pack(pady=10)
pb['value'] = 45      # 45%

# Indeterminate – animacja
pb2 = ttk.Progressbar(root,
    mode='indeterminate', length=300)
pb2.pack(pady=10)
pb2.start(10)         # animacja co 10ms
# pb2.stop()          # zatrzymaj
```

⚠ **ttk** nie ma parametrów `bg/fg` — wygląd kontroluje `ttk.Style()`. Mieszanie `tk.Button` z `ttk.Button` w jednej aplikacji daje nispójny wygląd — lepiej wybrać jeden zestaw.

Mini-projekt

Kompletna aplikacja GUI - klasa, layout, logika, menu

Struktura aplikacji - klasa

Menedżer kontaktów: dodawanie, lista, wyszukiwanie, zapis CSV

```
import tkinter as tk
from tkinter import ttk, messagebox, filedialog
import csv

class ContactApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Menedżer kontaktów")
        self.root.geometry("600x450")
        self.contacts = []

        self._create_menu()
        self._create_widgets()
        self._create_bindings()

    def _create_menu(self):
        menubar = tk.Menu(self.root)
        self.root.config(menu=menubar)
        file_menu = tk.Menu(menubar, tearoff=0)
        menubar.add_cascade(label="Plik", menu=file_menu)
        file_menu.add_command(label="Eksportuj CSV", command=self.export_csv)
        file_menu.add_separator()
        file_menu.add_command(label="Wyjście", command=self.root.quit)

    def _create_bindings(self):
        self.root.bind("<Return>", lambda e: self.add_contact())
        self.root.protocol("WM_DELETE_WINDOW", self._on_close)

    def _on_close(self):
        if messagebox.askokcancel("Wyjście", "Zamknąć aplikację?"):
            self.root.destroy()
```

Widgety i layout

```
def _create_widgets(self):
    # === Formularz (góra) ===
    form = ttk.LabelFrame(self.root, text="Nowy kontakt", padding=10)
    form.pack(fill=tk.X, padx=10, pady=5)

    ttk.Label(form, text="Imię:").grid(row=0, column=0, sticky="e", padx=5)
    self.name_var = tk.StringVar()
    ttk.Entry(form, textvariable=self.name_var, width=25).grid(row=0, column=1, padx=5)

    ttk.Label(form, text="Email:").grid(row=0, column=2, sticky="e", padx=5)
    self.email_var = tk.StringVar()
    ttk.Entry(form, textvariable=self.email_var, width=25).grid(row=0, column=3, padx=5)

    ttk.Button(form, text="Dodaj", command=self.add_contact).grid(row=0, column=4, padx=10)

    # === Wyszukiwanie ===
    search_frame = ttk.Frame(self.root)
    search_frame.pack(fill=tk.X, padx=10, pady=2)
    self.search_var = tk.StringVar()
    self.search_var.trace_add("write", lambda *a: self.filter_contacts())
    ttk.Label(search_frame, text="🔍").pack(side=tk.LEFT)
    ttk.Entry(search_frame, textvariable=self.search_var, width=30).pack(side=tk.LEFT, padx=5)

    # === Tabela (Treeview) ===
    cols = ("imie", "email")
    self.tree = ttk.Treeview(self.root, columns=cols, show="headings", height=12)
    self.tree.heading("imie", text="Imię")
    self.tree.heading("email", text="Email")
    self.tree.pack(fill=tk.BOTH, expand=True, padx=10, pady=5)

    # Status bar
    self.status = tk.StringVar(value="Kontaktów: 0")
    ttk.Label(self.root, textvariable=self.status).pack(anchor="w", padx=10)
```


Logika i zdarzenia

```
def add_contact(self):
    name = self.name_var.get().strip()
    email = self.email_var.get().strip()
    if not name or not email:
        messagebox.showwarning("Brak danych", "Wypełnij imię i email!")
        return
    self.contacts.append({"imie": name, "email": email})
    self._refresh_tree()
    self.name_var.set("")          # wyczyść formularz
    self.email_var.set("")

def filter_contacts(self):
    self._refresh_tree()

def _refresh_tree(self):
    self.tree.delete(*self.tree.get_children())
    query = self.search_var.get().lower()
    for c in self.contacts:
        if query in c["imie"].lower() or query in c["email"].lower():
            self.tree.insert("", tk.END, values=(c["imie"], c["email"]))
    self.status.set(f"Kontaktów: {len(self.tree.get_children())}")

def export_csv(self):
    path = filedialog.asksaveasfilename(defaultextension=".csv",
        filetypes=[("CSV", "*.csv")])
    if path:
        with open(path, "w", newline="", encoding="utf-8") as f:
            writer = csv.DictWriter(f, fieldnames=["imie", "email"])
            writer.writeheader()
            writer.writerows(self.contacts)
        messagebox.showinfo("Eksport", f"Zapisano {len(self.contacts)} kontaktów.")

if __name__ == "__main__":
    root = tk.Tk()
    app = ContactApp(root)
    root.mainloop()
```

Dobre praktyki Tkinter

Architektura kodu

- Klasa aplikacji — cały stan w self.*
- Osobne metody: `_create_widgets`, `_create_menu`
- Logika oddzielona od GUI (MVC/MVP)
- `if __name__ == '__main__':` na końcu
- Stałe (kolory, fonty) jako zmienne klasy
- Nie używaj `from tkinter import *`
- Frame do grupowania — hierarchia widgetów

Najczęstsze pułapki

- `time.sleep()` zamraża GUI → `after()`
- Mieszanie `pack/grid` w kontenerze → deadlock
- `PhotoImage` bez zmiennej → GC usuwa obraz
- `command=func()` wywołuje OD RAZU!
- `lambda` w pętli bez `n=i` → zła wartość
- Brak `mainloop()` → okno nie reaguje
- Ciężkie obliczenia w `mainloop` → `threading`

Tkinter - interfejsy graficzne

1	Tk() i mainloop()	okno główne + pętla zdarzeń — fundament każdej aplikacji
2	Widgety	Label, Button, Entry, Text, Check, Radio, Listbox, Scale
3	Layout: pack, grid, place	pack = liniowy, grid = tabela, place = absolutny
4	Zmienne Tk	StringVar, IntVar — powiązanie danych z widgetami
5	command i bind()	command = prosty callback, bind = zdarzenia z event
6	Canvas	grafika 2D, rysowanie, drag & drop, animacja z after()
7	ttk i Treeview	natywny wygląd, tabela danych, Notebook, Style
8	Klasa aplikacji	struktura OOP, separacja GUI i logiki, menu, dialogi

Następny wykład: Testowanie, narzędzia i co dalej